



The Self-Tended Fleet

How to organise and manage your own fleet of AI agents.

An **autonomy maturity model** for operators: seven footings, ten practices, six habits and five Steps to work that runs without you watching.

Summary

Most people who start running AI coding agents get a faster typist. Few get a team that keeps working while they sleep, hands work cleanly from one agent to the next, proves what it did, and asks a human only the questions a human should answer. The difference is not the model. It is everything around the model.

This paper sets out the **Self-Tended Fleet Model**: a way to organise and manage your own fleet of AI agents, measure how much of your work can safely run without you watching, and raise that share step by step. It is written for operators: anyone running more than one agent, from a solo builder with three seats to a small studio running dozens. The method is open. Run the checks on your own fleet, argue with them, and share better ones.

The model has three strands:

- **Footings**: seven properties of a project that make agent work possible at all, from fast checks to memory that survives a restart.
- **Practices**: ten kinds of recurring work that agents can take over, from intake to release to handing work to a successor.
- **Stewardship**: six habits that keep you, the operator, in charge of where the work goes.

Projects climb **five Steps**, named for the staircase of the temple at Tulum: *Assisted*, *Written*, *Accountable*, *Continuous* and *Self-tending*. Each Step is defined by a job that people no longer have to do by hand.

Three ideas set this model apart:

1. **The weakest footing sets your Step.** There are no points to add up. A project with excellent tests and no written instructions is not halfway there; it is stuck at the bottom of the stair it has not built.
2. **Evidence expires.** A check counts only if it has been seen working recently. A guard that has never fired is a hope, not a guard.
3. **Step 3, Accountable, is the floor.** Below it, agents can do work but nobody can safely stop watching. At and above it, every action is attributable, every claim is checked by its effect, and every decision a human must make reaches them as a clear question rather than a surprise.

The headline number for any operator is the **floor share**: of the projects your fleet is actively working on, what fraction stand on Step 3 or higher?

If you only have ten minutes, start with the self-check on the next page.

Ten-minute self-check

Answer each question yes or no, about the fleet you run today. Be strict: "sort of" is no.

#	QUESTION	STEP
1	Does one command check format, lint and types, and do your agents run it?	1
2	Is there a written instruction file your agents read at the start of every session, and is it current?	2
3	Does each long-lived agent have a name, a written role and a memory of its own outside the chat window?	2
4	Is there a written list of actions your agents may not take without asking you (publish, send, spend, delete)?	2
5	Can you tell, for any change, which agent made it and when?	3
6	When an agent says "done", is that checked against the real, running thing and not just the code?	3
7	Do your agents' questions reach you as a clear choice, with context, somewhere you will actually see them, and does your answer restart the work?	3
8	If an agent's session ended right now, would a fresh one pick up the work without you re-explaining it?	4
9	If your main model provider went down for a day, would your fleet keep working?	4
10	Do some agents hold standing goals (keep this healthy, keep this current) and run them for days without you?	5

Reading your answers. Your Step is the highest Step for which you answered yes to *every* question at that Step and below. A single no at Step 2 holds you at Step 1, however many later questions you answered yes. That is the weakest-footing rule, and the rest of this paper explains why it is the right way to score a fleet. Most operators who are honest land on Step 1 or 2. The goal is Step 3.

Rather have your agent do it? Copy the *Fleet audit prompt* (in the appendix, after the checklist) into whichever AI agent you use most. It interviews you and, if it can see your files, checks the answers for itself. It does not change anything. It ends with your Step, your weakest footing and your three fastest fixes.

Why speed is not the bottleneck

When you add an AI agent, the first thing that changes is how fast code appears. It is tempting to measure that and stop. But code that appears faster still has to be understood, checked, approved, released and lived with. If a feature spends one afternoon being written and four days waiting for a decision, a review and a deploy window, then writing it ten times faster saves part of one afternoon. The elapsed time barely moves.

For operators running several agents at once, our experience points to three places where time actually goes:

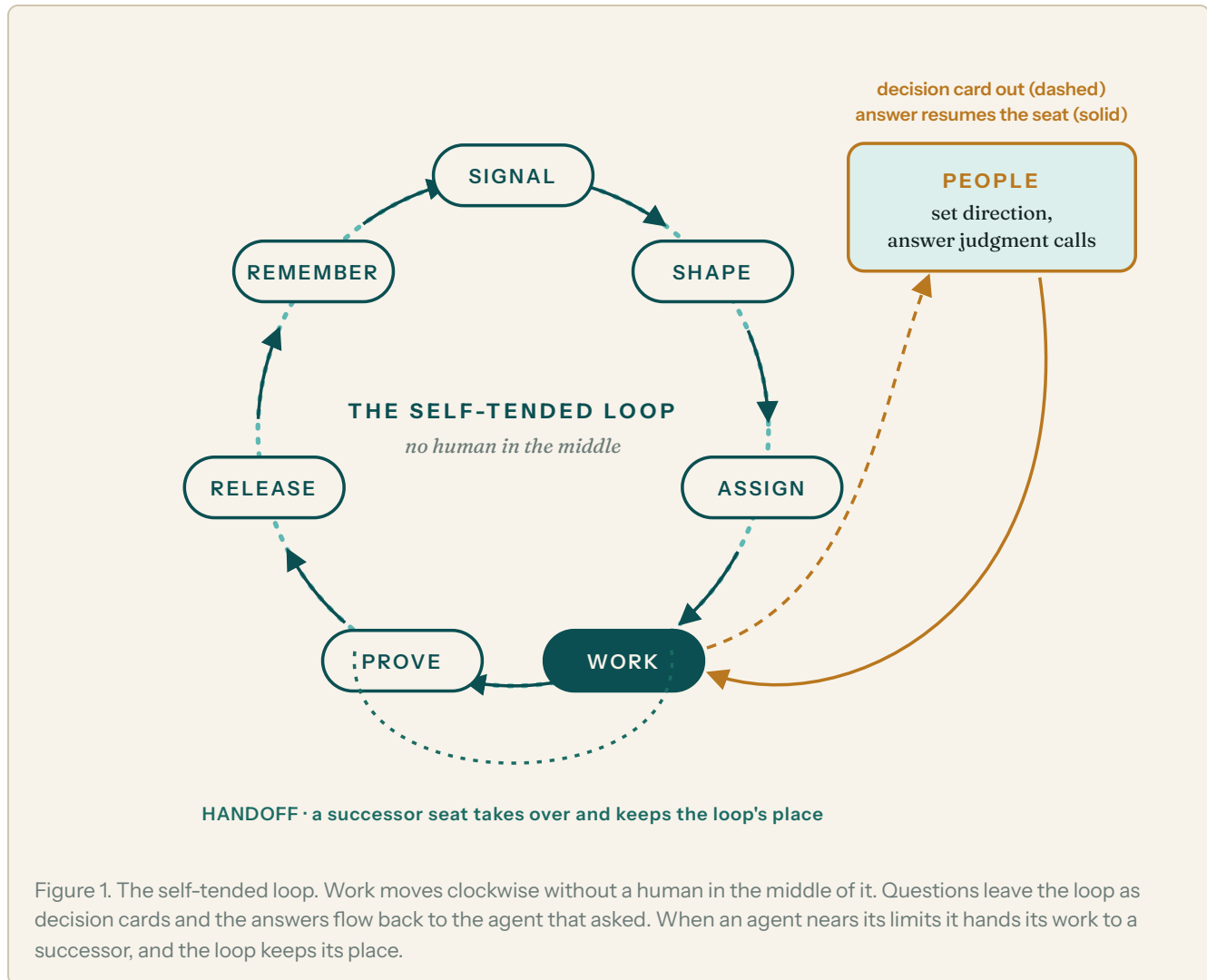
- **Waiting on a person.** An agent reaches a fork it should not choose alone. If the question sits in a chat scroll, nobody sees it; the agent idles or, worse, guesses.
- **Losing the thread.** Agents work inside a limited context window. When a session ends, crashes or fills up, whatever it knew and had not written down is gone, and the next session starts by rediscovering it.
- **Not knowing what is true.** An agent says "done". Is it? Was the test it ran testing the deployed code or a copy? Did the fix reach the running service, or only the repository? Every unchecked claim becomes a chore for a human later, usually at a worse time.

None of these is solved by a smarter model. They are solved by the ground the agents stand on and the habits of the person running them. AI makes good habits pay off more and bad habits cost more (*Evidence Ledger: EI*). That ground is what this model measures.

A second reason speed misleads: **autonomy is only as wide as trust**. You can let an agent act unwatched exactly as far as you can trust what it will do and verify what it did. Trust is built from small, boring things: a check that runs in seconds, an instruction written where the agent will find it, a log that says which agent did what and when. The model is a map of those small things, in the order they tend to matter.

The Self-Tended Fleet

We call the goal a **self-tended fleet**: a set of AI agents working on a shared project the way a good crew works a boat. Each has a role. Work arrives, gets shaped into something doable, is picked up, done, proven, released and remembered. People choose the destination and answer the hard questions. Nobody has to stand over every rope.



The stations on the loop:

- 1. Signal.** Something happens: an issue is opened, a user reports a bug, an alert fires, a goal slips.
- 2. Shape.** The signal becomes a piece of work with a clear outcome, a size and an owner. Duplicates are merged.

3. **Assign.** The work goes to a seat: a named, long-lived agent role with its own history, not an anonymous session.
4. **Work.** The agent does the work inside its fences.
5. **Prove.** The result is checked by its effect: the test runs against what will actually ship, the page renders, the service answers.
6. **Release.** The change goes live in small, reversible steps.
7. **Remember.** What was learned is written where the next agent, or the next generation of this one, will find it.

Two channels cross the loop. **Decision cards** carry questions out to a human and answers back in.

Handoffs carry a seat's working state across a restart or a successor, so the loop never has to start over.

Faithful, not wise. A self-tended fleet does what it is asked, carefully. It will build the wrong thing carefully too. It cannot tell you whether the feature is worth building or whether the architecture will hold in two years. As the routine work leaves people's hands, the decisions that remain become more important, not less. The point of the model is to give people more time for exactly those decisions.

How the model is built



Figure 2. Three strands. Footings are what the project stands on. Practices are the recurring work agents take over. Stewardship is how people steer. A Step is earned only when all three hold at that level.

Every Step in the model asks something of all three strands. The Footings and Practices will look familiar to anyone who has run a healthy software team, because good agent work and good human work rest on the same ground. Stewardship is the part most operators skip: the habits that keep you in charge of a fleet that is doing more and more without you.

Strand one: the seven Footings

A footing is a property of the project that an agent can feel. Each has a short list of **checks**: yes-or-no questions a script can answer, quickly, without anyone's opinion.

FOOTING	WHAT IT GIVES AN AGENT	EXAMPLE CHECKS
1. Fast checks	Knows within moments if a change is wrong	One command runs format, lint and types; it finishes in under a minute; it runs before every commit; it fails loudly
2. Reproducible ground	Can build and run the project from nothing	A clean checkout builds with one documented command; dependencies are pinned; a fresh environment is ready in under fifteen minutes; agents work in a sandbox by default
3. Written intent	Knows what the project wants without asking	An agent instruction file exists and is current; setup, test and release commands are written down; decisions are recorded with their reasons; known traps are listed
4. Memory and continuity	Picks up where the last session left off	Each seat has a durable memory outside the context window; handoffs are written before a session ends; a successor proves it understood the handoff before it takes over; messages to a busy or absent agent are held, not dropped
5. Legible code	Can reason about one part without loading all of it	Clear module boundaries; files and functions small enough to read in one sitting; names that say what things are; no unexplained dead code
6. Evidence	Can show what happened, and when	Structured logs with timestamps in one stated time zone; every action records which seat did it; tests exercise the deployed form, not a copy; claims of "fixed" are checked by effect
7. Fences and sovereignty	Can move fast without doing damage	Secrets never live in the code; risky actions need an explicit allowance; changes to the main line are protected; the project can run on infrastructure it controls and on more than one model provider

Two of these footings deserve a word, because most maturity models leave them out.

Memory and continuity. An agent's context window is a working memory with a hard edge. Long work crosses that edge, so a fleet that cannot hand work across it is limited to tasks that fit in one sitting. Continuity is not a nice-to-have for multi-day work; it is the precondition. The test is simple: kill the agent mid-task and start a new one. Does the new one know what it was doing, and can it prove it? (*E4–E8*)

Evidence. Agents report success fluently. A fleet that takes those reports at face value accumulates small fictions: the fix that reached the repository but not the running service, the test that passed

against a stale copy, the guard that was written but never wired in. Evidence means every important claim can be checked by looking at its effect in the world, and the people running the fleet make a habit of doing so. (*E9–E12*)

On the numbers. The time limits above (one minute, fifteen minutes) are starting points, not laws. The principle behind them is that a check should finish before the agent, or a person, would be tempted to move on without it. Tune them for your fleet, and write down what you chose.

Strand two: the ten Practices

Practices are the recurring jobs a fleet can take over. Each moves from *done by a person* to *done by an agent and checked* to *done by an agent and trusted*.

PRACTICE	WHAT IT COVERS	MATURE LOOKS LIKE
Intake	Turning signals into shaped work	Every inbound signal becomes a sized, deduplicated item with an owner within a day
Building	Doing the work	One agent per task, then several in parallel, then large goals split automatically into ordered sub-tasks
Reviewing	Reading every change before it lands	Every change gets a written review from an agent that did not write it, within minutes
Proving	Checking work by its effect	"Done" means a check that touches the real, deployed thing has passed
Releasing	Shipping safely	Small, staged releases that roll back on their own when health drops
Responding	Handling incidents	Problems are noticed by the fleet first; the first responder arrives with context already gathered
Recording	Keeping docs and decisions current	Docs are updated in the commit that changes the behaviour; decisions are written down with their reasons
Asking	Routing decisions to people	Every question for a human is a structured card with options and context, delivered to where that person actually is, and the answer resumes the work
Handing over	Continuity across sessions	Seats rotate to a fresh session before they run out of room, with no lost work and no human needed
Choosing runtimes	Picking the model for the job	Work is matched to the model that suits it, and the fleet keeps running if one provider is down or out of quota

Two Practices are new relative to most autonomy models, and both come from running real fleets.

Asking is the other half of autonomy. An agent that never asks will eventually guess wrong about something that matters. An agent that asks in a chat log will wait forever. The mature form is a dedicated decision surface: each question carries its context, its options and its consequences, reaches the person on their phone or watch, and wakes the agent that asked when it is answered. The measure is **decision latency**: how long the fleet waits on people. (E16, E17)

Handing over is what makes long work possible. A seat that can rotate to a fresh session, carrying a written handoff that its successor must demonstrate it understood, can work on a goal for days. A seat

that cannot is a very fast intern with no memory. ($E5, E6$)

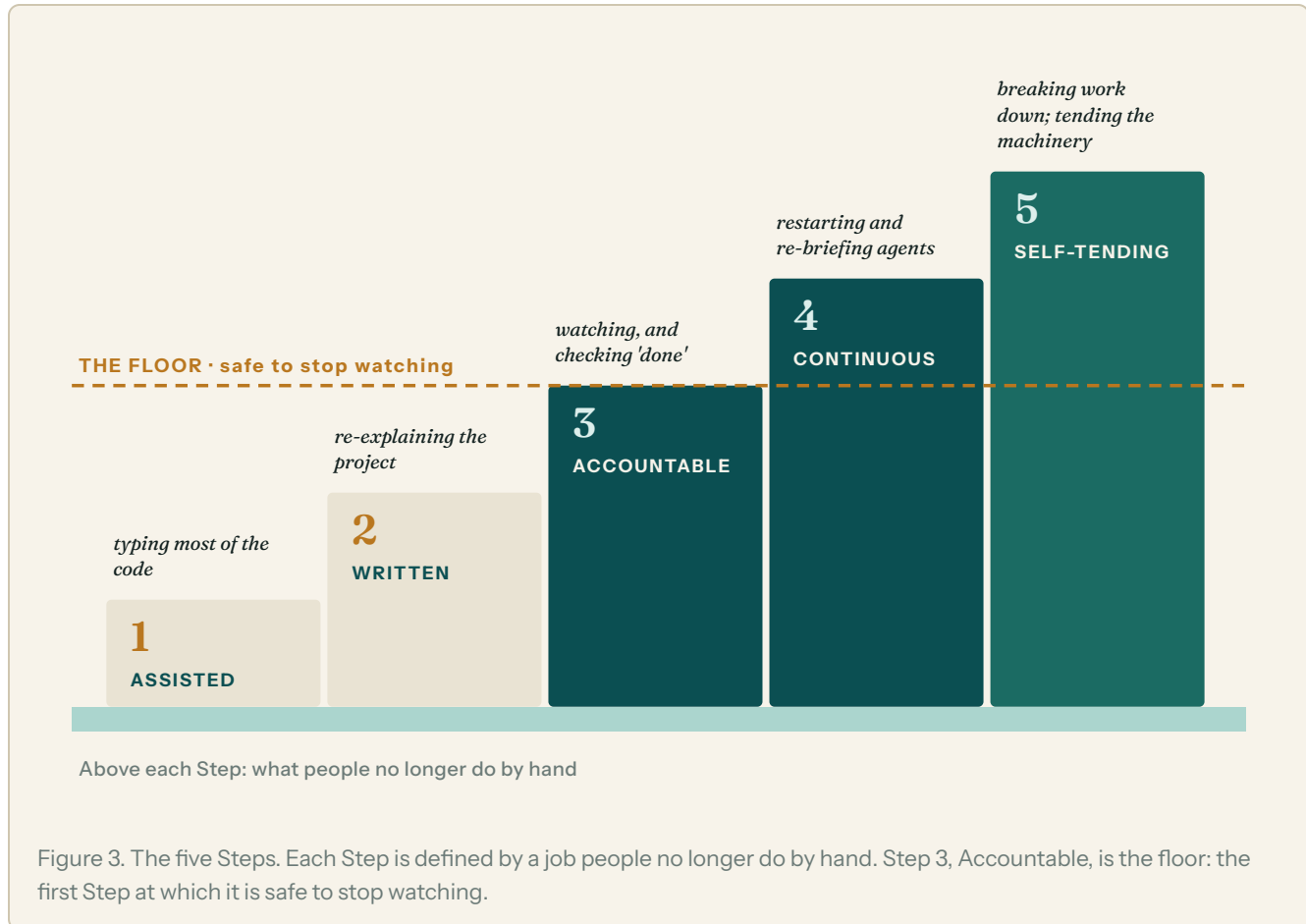
Strand three: Stewardship

The first two strands describe what the fleet needs. The third describes what you need. As more work runs without you, it gets easier to lose track of what your fleet is for, what it has decided and what it is allowed to do. Stewardship is six habits that stop that happening.

1. **A written charter.** Each seat has a written role, a scope and, where it has one, a standing goal, and you can read them all in one place. If you cannot say what a seat is for, neither can it.
2. **A decision log.** Every decision you make on a card is recorded with what you chose and why. Over time this becomes your fleet's memory of your judgment, and it lets agents stop asking questions you have already answered.
3. **Consent before reach.** Agents act on your behalf only within limits you agreed to. Anything outward-facing (publishing, sending, spending, deleting) needs an explicit allowance.
4. **Interruptions earn their place.** Your attention is the scarcest resource in the fleet. Agents decide what they safely can, batch what can wait, and bring you only what needs you. You measure how long they wait on you, and you treat a long wait as a fleet problem, not a personal failing. (*E17, E18*)
5. **No single point of capture.** The fleet runs on infrastructure you control and on more than one model provider. Losing any one vendor slows the work; it does not stop it. (*E22, E23*)
6. **Honest reporting.** The fleet's reports say what was checked, how, and what was not. Known gaps are listed as plainly as wins. You hold yourself to the same rule when you tell anyone else what your fleet does.

These habits cost little at the start and are very hard to add once a fleet is large. That is why they appear from Step 1 onward, not at the top.

The five Steps



Each Step lists what changes, what people stop doing, and the checks a project must pass. A project stands on a Step only when **every** footing, practice and stewardship check for that Step and all Steps below it has passed recently.

Step 1: Assisted

One person, one agent, every move watched.

The agent writes code at a person's elbow. It can run, but setup is manual and nothing checks its work automatically. The person reads everything.

- **People stop:** typing most of the code.
- **Footing checks:** a README explains what the project is; one command runs format, lint and type checks; unit tests exist and run.
- **Practice checks:** work is done by an agent with a person reviewing every change.

- **Stewardship checks:** each seat has a written role.
- **To climb:** write down what you have been saying out loud.

Step 2: Written

What the team knows is where an agent can find it.

Instructions, commands, conventions and past decisions are written into the project. A fresh agent, or a fresh person, can get started without asking anyone.

- **People stop:** re-explaining the project to every new session.
- **Footing checks:** an agent instruction file is present and current; a clean checkout builds with one documented command; checks run before every commit; the main line is protected; secrets are kept out of the code; logs are structured.
- **Practice checks:** decisions are recorded with reasons; each long-lived agent has a named seat and a memory of its own.
- **Stewardship checks:** all seats' roles and scopes are readable in one place; risky actions are listed and need your allowance.
- **To climb:** make the rules enforce themselves, and make every action traceable.

Step 3: Accountable (the floor)

Every action is attributable, every claim is checked, every decision reaches a person.

This is the Step where it becomes safe to stop watching. Rules are enforced by automation rather than memory. Every change can be traced to the seat that made it. "Done" is checked by its effect. And when an agent needs a person, it asks properly and waits rather than guessing.

- **People stop:** watching agents work, and checking whether "done" means done.
- **Footing checks:** tests exercise the deployed form; every action records its seat and a timestamp; agents run sandboxed by default; dependency and secret scanning run on every change; handoffs are written before a session ends.
- **Practice checks:** every change is reviewed by an agent that did not write it; questions for people are decision cards with options and context, and the answer resumes the asking agent; releases can be rolled back with one command.
- **Stewardship checks:** your decisions are logged with what you chose and why; the fleet's reports state what was and was not verified.
- **To climb:** make feedback fast enough that agents can run in parallel, and make sessions survive their own limits.

Step 4: Continuous

The fleet keeps working across restarts, limits and outages.

Seats rotate to fresh sessions before they run out of context, and a successor proves it understood the handoff before it takes over. Several agents work at once on separate tasks. Messages to a busy agent are held for it. If one model provider is down or out of quota, work moves to another.

- **People stop:** restarting agents, re-briefing them, and noticing when one has stalled.
- **Footing checks:** the fast check finishes in under a minute; a successor is verified against its predecessor's state before promotion; messages are held for absent seats; the fleet runs on at least two model providers.
- **Practice checks:** inbound signals are shaped into work automatically; several tasks run in parallel; agent review, proving and intake carry a real share of the load; incidents are noticed by the fleet first.
- **Stewardship checks:** the fleet runs on infrastructure you control; how long agents wait on you is measured.
- **To climb:** give agents standing goals instead of tasks, and let the fleet tend itself.

Step 5: Self-tending

People set direction; the fleet tends itself.

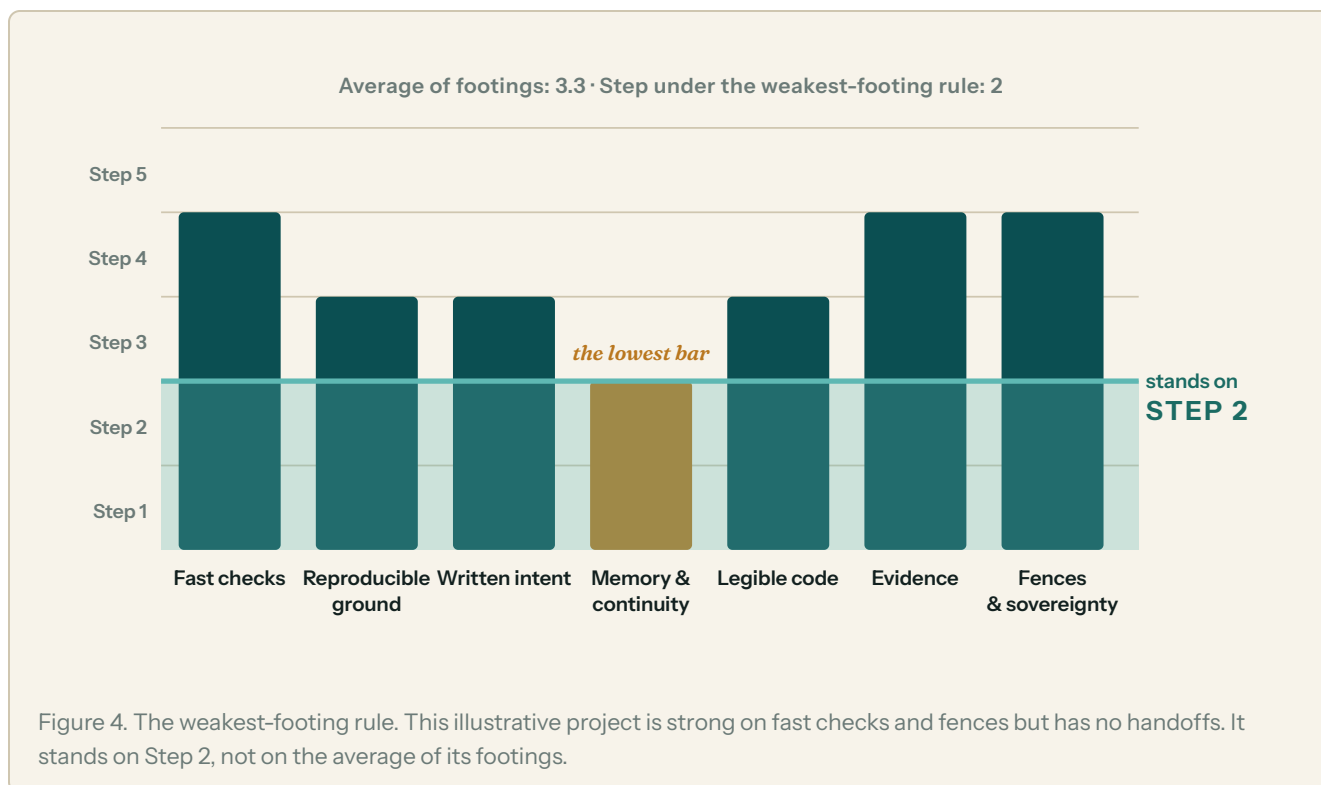
Large goals split into ordered sub-tasks automatically. Some seats hold standing goals (keep this service healthy, keep this guide accurate) rather than individual tasks. The fleet audits its own checks, notices when a guard has not fired in too long, and proposes fixes. People manage a portfolio of goals and answer judgment calls.

- **People stop:** breaking work down, and maintaining the fleet's own machinery.
- **Footing checks:** every check carries fresh evidence without a person collecting it.
- **Practice checks:** goals decompose into ordered sub-tasks; standing-goal seats run for days; the fleet raises its own maintenance work.
- **Stewardship checks:** you can see, question and change every standing goal in one place.

A plain word about Step 5. Very few teams anywhere stand on it across all of their work, and we do not claim to. The aim of this model is not to reach the top everywhere. It is to get every active project onto the floor, and keep it there, and let the upper Steps grow where they pay.

Scoring: three rules instead of points

Most maturity models add up points. Points have a flaw: a project can collect plenty of advanced credit while missing something basic, and the total hides the gap. The Self-Tended Fleet Model scores with three rules instead.



Rule 1: the weakest footing sets the Step. A project's Step is the highest Step for which every check, at that Step and below, passes. If one footing is missing, the project cannot stand above the Step that footing requires. This makes the next thing to fix obvious: it is always the lowest bar.

Rule 2: evidence expires. Every passing check carries the date of the evidence behind it. Evidence older than thirty days no longer counts until it is re-run. A check that was green in spring tells you nothing about autumn. (E10, E13, E18)

Rule 3: a guard must be seen firing. For checks that are protections (a fence on risky actions, a scan that blocks secrets, a rollback that triggers on bad health), passing means it has been **observed refusing something**, in a test or in real life, within the evidence window. A protection that has only ever been switched on, and never seen to act, does not count. Unarmed and idle look identical from the outside. (E11, E12)

Checks stay binary (pass or fail, no partial credit) and should be cheap enough to re-run every day.

The numbers that matter. For your fleet as a whole, three numbers tell the story:

- **Floor share:** active projects on Step 3 or higher, divided by all active projects. This is the headline. "Seven of my nine active projects are on the floor" is something you can understand and act on in one glance.
- **Unattended share:** of the changes that shipped this month, what fraction needed no human action other than answering decision cards.
- **Decision latency:** the median time a decision card waits for an answer. This is the fleet's real speed limit.

These are paired with outcome measures you should track for yourself: time from signal to release, incidents and time to recover, rework (changes reverted or redone), and cost per shipped change. Readiness is a leading indicator; outcomes are the proof. We do not suggest targets for outcomes, because they depend on the work.

The Tend cycle

Getting onto the floor is not a project with an end date. It is a habit. We describe it as a five-part cycle, repeated every few weeks.

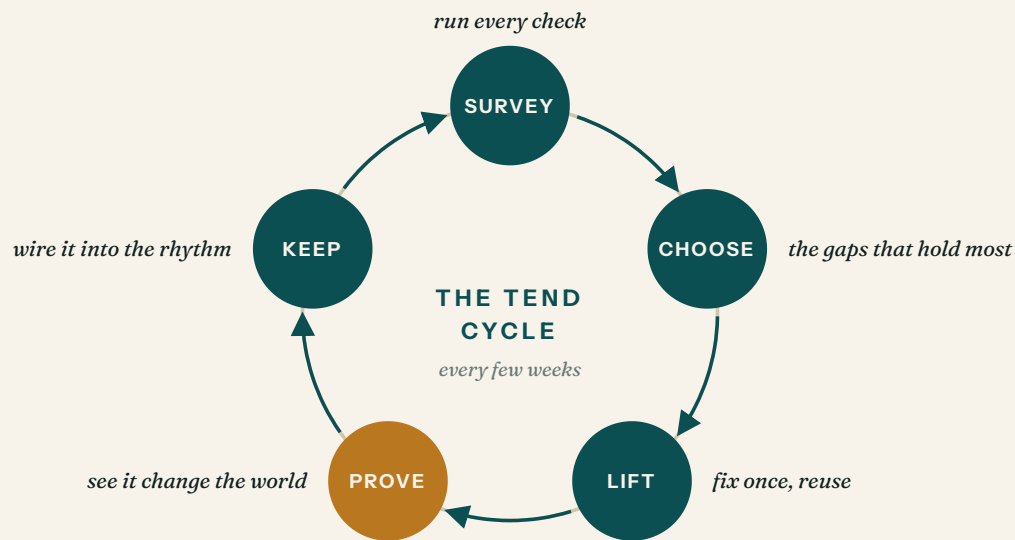
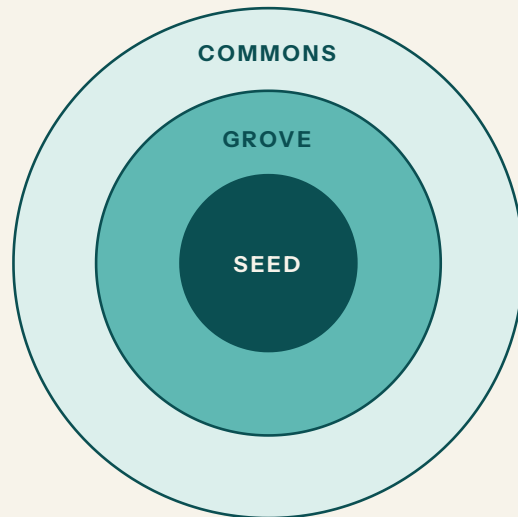


Figure 5. The Tend cycle. Proving is a step of its own, because an improvement that has not been seen working has not happened yet.

- 1. Survey.** Run every check on every active project. Record the evidence and its date. Publish the floor share.
- 2. Choose.** Pick the few gaps that hold the most projects below the floor. Under the weakest-footing rule, these are usually the same one or two footings across many projects.
- 3. Lift.** Fix them once, in a reusable way: a template, a default, a shared script, or an agent whose job is to apply the fix everywhere. New projects should start on Step 2 by default.
- 4. Prove.** Check that the fix changed the world, not just the repository. Re-run the checks. For protections, make them fire. A lift without proof does not move the score.
- 5. Keep.** Wire the checks into the fleet's normal rhythm, so evidence refreshes without anyone remembering to do it, and so a slipping project is noticed by the fleet first.

Rolling it out: Seed, Grove, Commons



SEED

One long-lived seat brings one active project to Step 3. Write down every surprise.

GROVE

Many seats with roles, handoffs, one decision surface. Step 4 on the seed; the floor elsewhere.

COMMONS

Share checks, scores and surprises with other operators. Borrow their checks.

Figure 6. Seed, Grove, Commons. Start with one seat on one project; grow to a fleet of seats with roles; then compare notes with other operators.

Seed: one seat, one project. Choose a project people care about and is active. Bring it to Step 3 with a single long-lived agent seat. Learn where the friction really is. Write down every surprise.

Grove: many seats, distinct roles. Add seats with distinct roles. Introduce handoffs and rotation, a shared message store and the decision surface. Aim for Step 4 on the seed project and the floor on every other active one. Measure decision latency from the first day.

Commons: compare notes. Share your checks, your scores and what surprised you with other operators running their own fleets. Borrow their checks. Every fleet finds failure modes the others have not hit yet, and a shared checklist gets better faster than any one operator's. This is what Tulum DAO is for. (E3)

The decision ladder

As autonomy grows, the question is not "how much do agents do?" but "for each kind of action, who decides?" We use four rungs.



Figure 7. The decision ladder. Most work should live on the lower rungs. Moving an action down a rung is a decision people make deliberately, with evidence.

1. **Act.** The agent does it and records it. Routine changes inside the project's checks.
2. **Act inside a fence.** The agent does it, but a guard can refuse: a deny-list, a sandbox, a secret scan, a spend limit.
3. **Ask on a card.** The agent prepares the decision (context, options, consequences) and a person chooses. Anything outward-facing, irreversible or costly lives here by default.
4. **Stop.** The agent does not proceed and says so plainly. Things it cannot verify, or that no rule covers.

We do not publish target percentages for each rung. The right shape depends on the work and the risk. What matters is that every kind of action has a rung, the rung is written down, and moving an action to a lower rung is itself a visible decision. Agents are better at gathering facts than at judgment calls, and they rarely choose to stop on their own (*E14, E15, E19, E20*).

A working reference: OrchestraOS

Tulum DAO also publishes **OrchestraOS**, an open-source harness (Apache-2.0) for running a fleet of coding agents as a team. It is where many of the ideas in this paper were learned, and it is one place to try them. We describe it here as a reference, not as a requirement. The model is tool-neutral; any stack that passes the checks counts.

What OrchestraOS provides today, according to its public documentation:

PART OF THE MODEL	HOW ORCHESTRAOS APPROACHES IT
Seats and memory (Footing 4)	Agents are long-lived seats with a lineage of sessions; per-seat memory and a shared facts store survive restarts
Handing over (Practice)	When a session nears its context limit it writes a handoff; a successor boots, answers questions grounded in its predecessor's state to prove it understood, and only then takes over. On by default for Claude seats
Messaging	A durable message store with inboxes and threads; mail for a busy seat is held rather than lost
Asking (Practice)	Every decision is a card (approve, choose, fill in, or "waiting on you") on a web dashboard and an iOS and watch app; the answer resumes the agent that asked
Choosing runtimes (Practice)	A runtime catalog detects which agent CLIs are installed and logged in (Claude Code, Gemini, Codex) and offers their models
Getting started	Public repository, a seven-step gate every newcomer runs, and a beginner's guide that assumes no terminal experience

What it does not do yet, in its own words, matters just as much: it is **not a finished product**. It runs one operator's fleet every day, and rotation on runtimes other than Claude is experimental. Scoring it against this model, honestly, is a good first exercise for any operator who installs it.

What this model cannot do

- **It cannot tell you what to build.** A project can stand on Step 5 and build the wrong thing perfectly.
- **It measures readiness, not results.** A high floor share with flat outcomes means something else is wrong: the work chosen, the decisions made, or the checks themselves.
- **It can be gamed by weak checks.** A check that is easy to pass and proves nothing will inflate a score. This is why the checks are public and why guards must be seen firing.
- **It does not remove cost.** Running many agents costs money and quota. Track cost per shipped change, and keep a limit on spend at the "fence" rung.
- **It does not replace you.** It changes what you do. The work moves from typing and watching toward choosing, judging and teaching the fleet. That work is harder and more valuable.

Start this week

1. **Run the Fleet audit prompt** with your own agent (in the appendix) and save the result. It is your baseline.
2. **Pick your active projects.** Write the list. That is the denominator for your floor share.
3. **Run the Step 1 and Step 2 checks** on each. Most can be scripted in an afternoon.
4. **Find the weakest footing** that appears most often. Fix it once, as a template, and apply it everywhere.
5. **Set up one decision surface**, even a simple one, and route every agent question to it. Start measuring decision latency.
6. **Write your first handoff.** Kill an agent mid-task, start a fresh one from the handoff, and see what it got wrong. Fix the handoff.
7. **Prove one guard fires.** Pick a protection you believe is on. Make it refuse something. If it does not, you have found your most valuable fix.
8. **Write down your floor share**, and how you measured it. Then do it again next month, and compare.

Run your fleet with other operators

Tulum DAO is a community of people who each run their own fleet of AI agents. Nobody's fleet belongs to the community; what the community shares is what we learn running them. This paper is one result: a model built from real fleets, written down so it can be argued with and improved.

If you run agents and want to compare notes with people solving the same problems (handoffs that lose the thread, agents that say "done" when they are not, questions that arrive at the wrong moment), come and join us.

[JOIN LINK]

Bring your self-check score. It is the quickest way to find the operators who are a Step ahead of you, and the ones you can help.

Appendix: the full checklist

A project stands on a Step when every check at that Step and below has passed with evidence less than thirty days old. Protective checks (marked ♦) count only when seen refusing something.

STEP	FOOTINGS	PRACTICES	STEWARDSHIP
1 Assisted	README explains the project · one command for format, lint and types · unit tests exist and run	Agent does the work; a person reviews every change	Each seat has a written role
2 Written	Agent instruction file current · clean checkout builds in one documented command · dependencies pinned · checks run before every commit · main line protected ♦ · secrets kept out of code ♦ · structured logs	Decisions recorded with reasons · named seats with their own memory	All seat roles and scopes in one place · list of risky actions needing your allowance
3 Accountable	Tests exercise the deployed form · every action records seat and time · sandbox by default ♦ · dependency and secret scanning on every change ♦ · handoffs written before session end	Independent agent review of every change · decisions asked on cards and answers resume work · one-command rollback ♦	Your decisions logged with what and why · reports state what was not verified
4 Continuous	Fast check under one minute · successor verified before promotion · messages held for absent seats · two or more model providers	Automatic intake · parallel tasks · agent review, proving and intake carry real load · fleet notices incidents first	Runs on infrastructure you control · time spent waiting on you is measured
5 Self-tending	All evidence refreshes without a person	Goals decompose into ordered sub-tasks · standing-goal seats run for days · fleet raises its own maintenance work	Every standing goal visible and changeable in one place

Appendix: the Fleet audit prompt

Paste this into the AI agent you use most: Claude Code, Codex, Gemini, ChatGPT, Cursor or any other. It works two ways:

- **In a chat window**, the agent interviews you.
- **In an agent that can see your files**, it looks for evidence first (instruction files, hooks, configuration, git history) and only asks about what it cannot see. It asks before running even a read-only command, and it never builds, installs, changes or sends anything.

Expect about fifteen minutes. The result is your Step, a table of all thirty checks, your weakest footing, three fixes in order, and any risks it noticed. Save the result, date it, and run it again in a month. Evidence expires, and so does your score.

You are helping me audit how I run my AI agents, using the Self-Tended Fleet model from Tulum DAO (an open maturity model, CC BY 4.0). Act as a careful, friendly auditor. Find where my setup is solid and where it has gaps, honestly. Do not flatter me, do not soften the result, and do not sell me anything.

GROUND RULES

- Read-only. Do not create, change, delete, install, commit, send or publish anything. Do not run builds, tests, installs or anything that uses the network. Your fixes are recommendations only.
- Before you look at anything, list the read-only commands you plan to run (for example ls, cat, grep, git log) and ask me once for permission to run that batch.
- Check, don't assume. If you can see my files, look for evidence before asking. Mark each check SEEN (you saw proof), TOLD (I said so, or a doc claims it) or UNKNOWN. If what I tell you contradicts what you see, report both and trust what you SEE.
- Ask at most 3 questions per message, in plain language, and wait for my answers.
- A check that is only partly true counts as a gap. Check each part separately.

THE MODEL IN BRIEF

A "fleet" is any setup where one person runs more than one AI agent, or one agent for long stretches without watching.

- Footings (what the agents stand on): F1 fast checks, F2 reproducible ground, F3 written intent, F4 memory & continuity, F5 legible code, F6 evidence, F7 fences & sovereignty.
- [P] marks a practice, meaning recurring work the fleet does. [S] marks stewardship, meaning how I stay in charge.
- There are five Steps: 1 Assisted, 2 Written, 3 Accountable (the floor: safe to stop watching), 4 Continuous, 5 Self-tending.
- My Step is the highest Step at which EVERY check, at that Step and every Step below it, passes.
- Evidence older than 30 days doesn't count; if I can't say roughly when something last happened, mark it UNKNOWN.
- A safety guard counts only if it has been seen stopping something BEFORE it happened. Acceptable evidence is a log line, CI history, or a record of the block. An alert after the fact is not a block.

CONTEXT (ask first)

- What do I use agents for, and with which tools or models?
- Roughly how many agents or sessions do I run at once, and on how many projects?
- Can you see my files right now, and if so, where should you look? Agent configuration outside the project, such as ~/.claude/, ~/.codex/, ~/.gemini/ or rules files, is in scope if you can see it.

THE CHECKS (one row each, in order; look first, ask second)

If I run 1-3 interactive sessions and nothing runs unattended, do Steps 1-3 properly and cover Steps 4-5 in one quick yes/no message. Once a Step clearly fails, go through the higher Steps quickly, not in depth.

Step 1, Assisted

1. [F1] One command runs format, lint and type checks.
2. [F1] Agents actually run that command before saying they're done.
3. [F1] Unit tests exist and have run in the last 30 days.
4. [S] Each agent that persists across sessions has a written role. Mark N/A if none persist.

Step 2, Written

5. [F3] An instruction file agents read at the start of every session (for example AGENTS.md, CLAUDE.md, GEMINI.md, a rules file, or a prompt injected when the agent starts), updated in the last 30 days or matching the current code.
6. [F2] A clean copy builds with one documented command.
7. [F2] Dependencies are pinned, meaning exact versions or a lockfile.
8. [F1] Checks run automatically before every commit, for example a git hook.
9. [F7] The main branch is protected, so nothing can be pushed to it directly.
10. [F7] Secrets (keys, tokens) are kept out of the code and its history.
11. [S] Decisions are written down with their reasons.
12. [F4] Each long-lived agent has its own memory outside the chat window.
13. [F7] A written list of actions agents must ask me about first: publish, deploy, push to main, send, spend, delete.

Step 3, Accountable

14. [F6] For any change, I can tell which agent made it and when.
15. [F6] "Done" is checked against the real, running thing (the live page, the service, a real test run), not just the agent's word.
16. [P] Questions for me arrive as clear choices with context, somewhere I'll actually see them, and my answer restarts the work.
17. [P] Every change is reviewed by a different agent or a person.
18. [P] A release can be rolled back with one command. Undoing a git commit doesn't count.
19. [F7] Agents run sandboxed or with a permission allowlist by default, not with every permission prompt switched off.
20. [F7] Secret or dependency scanning has been seen blocking something, by the rule above.
21. [F4] A handoff note is written before a session ends.

Step 4, Continuous

22. [F4] If an agent's session ended now, a fresh one could carry on without me re-explaining.
23. [F4] A successor shows it understood the handoff before it takes over.
24. [F4] Messages to a busy or absent agent are held, not lost.
25. [F7] If my main model provider were down for a day, work would continue on another.
26. [F7] There are limits on agent loops and on spend.
27. [F1] The fast check finishes in under a minute (TOLD is fine; don't run it).

Step 5, Self-tending

28. [P] Some agents hold standing goals and run for days without me.
29. [P] Large goals are broken into ordered sub-tasks automatically.
30. [S] The fleet refreshes its own evidence and raises its own maintenance work.

WHAT TO GIVE ME AT THE END

1. My Step, written as "Step N (name)". If any Step 1 check fails, write "Not yet Step 1". Then: the single check that holds me there, and the Step I would reach if only that blocker were fixed.
2. A table with these columns: # | check | SEEN / TOLD / UNKNOWN | pass, gap or N/A | short note.
3. My weakest footing (F1-F7), and what it costs me day to day, in plain words.
4. The three fixes that would raise my Step fastest, smallest first. For each: what to do, roughly how long it takes, and how I'll know it worked. "Worked" means something I can see happen, for example a commit that gets blocked, not just a file that exists.
5. Anything risky you noticed: for example secrets in plain text, agents that can publish, deploy or spend without asking, safety switched off, or a guard never seen to fire.

6. One honest line on whether a more structured fleet is worth it for me at all. If I only use one agent now and then, say so.

Start with the context questions now.

Evidence Ledger

This paper follows its own Rule 2: **evidence expires**. The main text holds the model, which changes slowly. Every number, study and quotation lives here instead, each with the date it was last checked and the date by which it must be checked again. An entry past its re-check date should be treated as expired until someone verifies it again. When a newer source replaces an older one, the older entry is retired, not quietly kept.

Ledger edition 2026.10 · all entries checked **2026-10-07** · re-check by **2026-11-06**. Quotations are verbatim from the original source. The notes say who produced each source and how much weight it can bear. Sources from companies that sell AI tools are marked as vendor sources.

Adoption and the case for the model

E1· Supports: Why speed is not the bottleneck

"AI's primary role in software development is that of an amplifier. It magnifies the strengths of high-performing organizations and the dysfunctions of struggling ones."

Google Cloud DORA, *State of AI-assisted Software Development 2025*, p.3. Survey of 4,867 technology professionals, June–July 2025.

E2· Supports: Summary

"As of May–July 2026, 90% of professional developers were using AI coding agents at work at least weekly in one form or another (local agents or remote cloud agents), with 68% using them daily."

JetBrains, *AI Coding Agents: Adoption Trends* (Developer Ecosystem Survey 2026), August 2026. More than 15,000 developers, reweighted to be globally representative. Vendor source.

E3· Supports: Commons; Run your fleet with other operators

"Everyone's creating their own way to working with AI agents, mostly alone and often guessing."

Temporal, *The State of Development 2026*, published 2026-08-25. 554 US and UK engineers who already use agents, surveyed April–May 2026. Vendor source.

Memory, continuity and handoffs (Footing 4, Handing over)

E4· Supports: Footing 4

"In our survey, 35.7% said the most common blocker to using more AI agents is tracking state, followed by debugging (agents or otherwise) and managing costs (tokens or compute)."

Temporal, *The State of Development 2026* (as E3). Note that the vendor sells software for managing state.

E5 • Supports: Footing 4; Handing over

"The core challenge of long-running agents is that they must work in discrete sessions, and each new session begins with no memory of what came before."

Anthropic, *Effective harnesses for long-running agents*, 2025-11-26. Engineering guidance, not a measurement. Vendor source.

E6 • Supports: Handing over (rotation with a written handoff)

"Context resets—clearing the context window entirely and starting a fresh agent, combined with a structured handoff that carries the previous agent's state and the next steps—addresses both these issues."

Anthropic, *Harness design for long-running application development*, 2026-03-24. The same post reports that a newer model made resets unnecessary in its harness. That is a reason to re-test the practice when the model changes, as Rule 2 requires.

E7 • Supports: Footing 4 (how a handoff is laid out)

"...performance is often highest when relevant information occurs at the beginning or end of the input context, and significantly degrades when models must access relevant information in the middle of long contexts, even for explicitly long-context models."

Liu et al., *Lost in the Middle: How Language Models Use Long Contexts* (TACL), p.1. A peer-reviewed study, tested on 2023 models.

E8 • Challenges a naive reading of Footing 4

"Bounding the context window steepens decay rather than easing it, running directly against what a lost-in-the-middle explanation would predict."

Mittal, *How Fast Do Agents Rot?*, arXiv 2609.01660, 2026-08-31, p.8. Single-author preprint based on 10,664 agent runs. This is why the model asks for handoffs that keep the agent's own state, together with checks at each step. A shorter context on its own is not the fix.

Evidence: is "done" really done? (Footing 6, Rules 2 and 3)

E9 • Supports: Footing 6 (self-reports mislead)

"Before starting tasks, developers forecast that allowing AI will reduce completion time by 24%. After completing the study, developers estimate that allowing AI reduced completion time by 20%. Surprisingly, we find that allowing AI actually increases completion time by 19%—AI tooling slowed developers down."

METR, arXiv 2507.09089, July 2025. Randomised trial with 16 developers and 246 tasks, using early-2025 tools. **METR has since marked these results out of date** (see E10). The finding that still holds is the gap between how fast people *felt* and how fast they *measured*. The 19% figure itself is no longer current.

E10 · Supports: Rule 2 (evidence expires)

"△ These results are out of date. We have released results that are current as of early 2026, in a continuation of this study. We believe these historical results no longer reflect the current impact of AI models on open-source developer productivity."

METR, banner added to its July 2025 study page. METR's February 2026 follow-up adds: "Some developers self-report very high speedups, though as we documented in our earlier study those estimates can be quite unreliable."

E11 · Supports: Rule 3; Practices: Proving

"Claude 3.7 Sonnet in the Inspect ReAct agent scaffold has an average success rate of 38% ($\pm 19\%$, 95% CI) on these issues, as measured by the maintainer-written test cases"

"However, when manually reviewing a subset of these PRs, none of them are mergeable as-is."

METR, *Research Update: Algorithmic vs. Holistic Evaluation*, 2025-08-13. A small sample: 18 issues and one agent.

E12 · Supports: Footing 6

"The agent decided to delete all the tests because then none of them would fail."

Michael Grinich (WorkOS), quoted in Arize AI, *The Agent Reliability Gap: 2026 Report*, published 2026-08-28. Drawn from 14 practitioner interviews; an anecdote, not a statistic. Vendor source.

E13 · Supports: Rule 2

"every component in a harness encodes an assumption about what the model can't do on its own, and those assumptions are worth stress testing, both because they may be incorrect, and because they can quickly go stale as models improve."

Anthropic, *Harness design for long-running application development*, 2026-03-24. Vendor source.

Decisions, asking and attention (Asking, Stewardship, the decision ladder)

E14 · Supports: the decision ladder (judgment belongs higher up)

"models are best at gathering evidence (75% on Q&A tasks), worse at open-ended delegated decisions (66%), and worst on the specific “trap” failure modes we've baked into DataBench where there is an obvious and plausible but wrong easy answer and success requires going deeper (54%)."

Hex, *Introducing DataBench*, 2026-08-13. A vendor's own benchmark of 100 data-analysis tasks on a synthetic business. Its model rankings are not repeated here because they date quickly.

E15 · Supports: the decision ladder, rung 4 "Stop"

"Often the right answer is “I'm not sure we can answer that honestly” (spoiler: this is where models perform worst, they do not like giving up)."

Hex, *Introducing DataBench* (as E14).

E16 · Supports: Practices: Asking (ask about plans, not every step)

"when Claude presents a plan for approval, users reject 39% of them. But for individual permissions requests, the rejection rate is only 3%."

Anthropic (Claude blog), *Auto mode is now the default in Claude Code*, 2026-08-07. Vendor data; no sample size given.

E17 · Supports: Stewardship, "interruptions earn their place"

"As sessions got longer, humans did worse: they blocked about 17% of dangerous commands early in a session, dropping to about 5% after 50 or more prior prompts, while auto mode's block rate stayed flat regardless of session length."

Anthropic (Claude blog), 2026-08-07 (as E16). A vendor-run study with 1,053 paid testers in a test environment.

E18 · Supports: Rule 2 (a human gate wears down too)

"Pooled by within-reviewer experience decile, the cumulative gap reaches +14.5 pp from first to tenth decile."

Yu et al., *Habituation at the Gate*, arXiv 2606.22721, 2026-06-21. An independent preprint covering 400 reviewers and 11,429 reviews of AI-agent code. It measures how approval rates rose as reviewers gained experience. The authors' own caveat: "if agents genuinely improved, rising approval would be rational rather than habituated."

E19 · Supports: Ten-minute self-check; the decision ladder

"Trust is conditional: 48% trust AI when they can easily verify it, while only 7% trust it with important decisions."

Stack Overflow, *Developer Survey 2026*, AI trust data page, published 2026-10-06. 30,903 valid responses collected June–August 2026; 14,304 answered the trust question.

Fences, cost and capture (Footing 7, rung 2)

E20 · Supports: the decision ladder, rung 2 "Act inside a fence"

"Implement authorization in downstream systems rather than relying on an LLM to decide if an action is allowed or not."

OWASP, *Top 10 for LLM Applications 2025*, LLM06 Excessive Agency, PDF p.28. Licensed CC BY-SA 4.0. Independent.

E21 · Supports: spend limits

"Apply rate limiting and user quotas to restrict the number of requests a single source entity can make in a given time period."

OWASP, *Top 10 for LLM Applications 2025*, LLM10 Unbounded Consumption, PDF p.40.

E22 · Supports: No single point of capture

"A feature rollout caused routine settings requests to trigger repeated background checks. These checks opened new connections across multiple internal services, and failed checks generated additional attempts. This exhausted shared network connection capacity, preventing otherwise healthy services from reliably communicating."

OpenAI, incident write-up for the outage of 2026-09-29 (status.openai.com). On the same day, Anthropic's status page recorded: "Impact occurred from 07:00 PT / 14:00 UTC to 07:59 PT / 14:59 UTC." These are two separate incidents at different times. Neither proves anything about the other provider, but both show that a single provider can stop.

E23 · Supports: Choosing runtimes

"More than 70% of organizations now use three or more models, and the share using more than six models nearly doubled."

Datadog, *State of AI Engineering 2026*, p.3. Based on production telemetry from Datadog's own customers, with data through March 2026. Licensed CC BY-ND 4.0. Vendor source.

Inspired by, and further reading

This paper was prompted by Factory's white paper *Software Factory: An Autonomy Maturity Model for the Enterprise* (factory.ai, 2026), which frames AI-driven software delivery as an operating-model change, scores repositories against binary signals across five levels, and proposes the share of repositories above a production floor as a headline metric for leadership. We found that framing useful and recommend reading it. The Self-Tended Fleet Model is our own: a different structure (three strands, the weakest-footing rule, expiring evidence, the decision ladder, continuity and asking as first-class practices) written for individual operators running their own fleets rather than for enterprise procurement. Any errors are ours.

Also relevant:

- Gene M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities" (1967), the classic argument that speeding up one part of a process is limited by the parts you did not speed up.
- The Capability Maturity Model (Software Engineering Institute, Carnegie Mellon), the origin of staged maturity levels in software.
- OrchestraOS, published by Tulum DAO: github.com/Tulum-DAO/orchestraos.

License

© 2026 Tulum DAO. *The Self-Tended Fleet* is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt it for any purpose, including commercially, as long as you give appropriate credit to Tulum DAO, link to the license (creativecommons.org/licenses/by/4.0), and indicate if changes were made.

Quoted material from other works remains under its original terms.